

Siempre que se proporcione la atribución adecuada, Google otorga permiso a reproducir las tablas y figuras de este documento únicamente para uso periodístico o obras académicas.

La atención es todo lo que necesitas

Ashish Vaswani *
Cerebro de Google
avaswani@google.com

Noam Shazer *
Cerebro de Google
noam@google.com

Niki Parmar *
Investigación de Google
nikip@google.com

Jakob Uszkoreit *
Investigación de Google
uszk@google.com

Lion Jones *
Investigación de Google
llion@google.com

Aidan N. Gómez *
Universidad de Toronto
aidan@cs.toronto.edu

Lukasz Kaiser *
Cerebro de Google
lukaszkaizer@google.com

Illia Polosukhin *‡
illia.polosukhin@gmail.com

Resumen

Los modelos de transducción de secuencias dominantes se basan en complejos, recurrentes o redes neuronales convolucionales que incluyen un codificador y un decodificador. ¿El mejor los modelos ejecutantes también conectan el codificador y el decodificador a través de una atención mecanismo. Proponemos una nueva arquitectura de red simple, la Transformer, basado únicamente en los mecanismos de atención, prescindiendo de la recurrencia y las convoluciones en su totalidad. Los experimentos realizados con dos tareas de traducción automática muestran que estos modelos: ser superior en calidad y, al mismo tiempo, ser más paralelizable y requerir significativamente menos tiempo para entrenar. Nuestro modelo obtiene 28,4 BLEU en la WMT 2014 en inglés Tarea de traducción al alemán, que mejora con respecto a los mejores resultados existentes, que incluye conjuntos, de más de 2 BLEU. Sobre la tarea de traducción del inglés al francés del WMT 2014, nuestro modelo establece una nueva puntuación BLEU de última generación para un solo modelo de 41,8 después formación durante 3,5 días en ocho GPU, una pequeña fracción de los costes de formación del mejores modelos de la literatura. Mostramos que el Transformer se generaliza bien a otras tareas aplicándolo correctamente a la circunscripción inglesa, analizando ambas con datos de entrenamiento grandes y limitados.

* Contribución igualitaria. El orden de los anuncios es aleatorio. Jakob propuso reemplazar las RNN por la autoatención y comenzó el esfuerzo por evaluar esta idea. Ashish, con Illia, diseñó e implementó los primeros modelos Transformer y ha participado de manera crucial en todos los aspectos de este trabajo. Noam propuso una atención escalonada al producto, multicabezal atención y representación de posiciones sin parámetros y se convirtió en la otra persona involucrada en casi todos detalles. Niki diseñó, implementó, ajustó y evaluó innumerables variantes de modelos en nuestra base de código original y tensor2/tensor. Lion también experimentó con variantes de modelos novedosas, fue responsable de nuestra base de código inicial y inferencia y visualizaciones eficientes. Lukasz y Aidan pasaron incontables y largos días diseñando varias partes de y implementar tensor2tensor, reemplazar nuestra base de código anterior, mejorar en gran medida los resultados y acelerar enormemente nuestra investigación.

† Trabajo realizado mientras estaba en Google Brain.

‡ Trabajo realizado mientras trabajaba en Google Research.

1 Introducción

Redes neuronales recurrentes, memoria a corto plazo [1] y recurrente cerrada [2] [3] redes neuronales en particular, se han establecido firmemente como enfoques de vanguardia en el modelado de secuencias y problemas de transducción como la modelización del lenguaje y la traducción automática [4, 5]. Numerosos [6, 7] Desde entonces, los esfuerzos han seguido ampliando los límites de los modelos lingüísticos recurrentes y del codificador-decodificador arquitecturas [38, 24, 15].

Los modelos recurrentes suelen factorizar el cálculo a lo largo de las posiciones de los símbolos de la entrada y la salida. secuencias. Al alinear las posiciones con los pasos del tiempo de cálculo, generan una secuencia de elementos ocultos h_t , en función del estado oculto anterior h_{t-1} y la entrada para la posición t . Esto inherentemente la naturaleza secuencial impide la paralelización dentro de los ejemplos de entrenamiento, lo que se vuelve fundamental a mayor tiempo longitudes de secuencia, ya que las restricciones de memoria limitan el procesamiento por lotes entre los ejemplos. Un trabajo reciente ha logrado mejoras significativas en la eficiencia computacional mediante trucos de factorización [8] y condicional [9] [10], al tiempo que mejora el rendimiento del modelo en el caso de este último. Lo fundamental Sin embargo, se mantiene la restricción de la computación secuencial.

Los mecanismos de atención se han convertido en una parte integral de la modelización y transducción de secuencias convincentes modelos de acción en varias tareas, lo que permite modelar dependencias sin importar su distancia en las secuencias de entrada o salida [11, 12, 19]. En todos los casos, excepto en un [13] pocos [14], sin embargo, dichos mecanismos de atención se utilizan junto con una red recurrente.

En este trabajo proponemos el Transformer, una arquitectura modelo que evita la recurrencia y, en cambio, confiando completamente en un mecanismo de atención para establecer dependencias globales entre la entrada y la salida. El Transformer permite una paralelización significativamente mayor y puede alcanzar un nuevo estado del arte en calidad de traducción después de haber recibido formación durante tan solo doce horas en ocho GPU P100.

2 Antecedentes

El objetivo de reducir la computación secuencial también constituye la base de la GPU neuronal extendida. [15] de TeNet [16] y CoNet [17], todos los cuales utilizan redes neuronales convolucionales como construcción básica bloque, calculando las representaciones ocultas en paralelo para todas las posiciones de entrada y salida. En estos modelos, aumenta el número de operaciones necesarias para relacionar señales desde dos posiciones arbitrarias de entrada o salida en la distancia entre las posiciones, linealmente para ConvS2s y logarítmicamente para ByteNet. Esto hace es más difícil aprender las dependencias entre posiciones distantes [18]. En el Transformer está reducido a un número constante de operaciones, aunque a costa de la reducción de la resolución efectiva debida para promediar las posiciones ponderadas por la atención, un efecto que contrarrestamos con la atención con múltiples cabezas como descrito en la sección 3.2.

La autoatención, a veces llamada intraatención, es un mecanismo de atención que relaciona diferentes posiciones. de una sola secuencia para calcular una representación de la secuencia. La autoatención ha sido utilizado con éxito en una variedad de tareas que incluyen la comprensión lectora, el resumen abstracto, representación textual de oraciones independientes de la tarea y el aprendizaje [4, 27, 28, 22].

Las redes de memoria de extremo a extremo se basan en un mecanismo de atención recurrente en lugar de en una secuencia se alinearon con la recurrencia y se ha demostrado que funcionan bien en la respuesta a preguntas en un lenguaje sencillo y tareas de modelado del lenguaje [34].

Sin embargo, hasta donde sabemos, el Transformer es el primer modelo de transducción que depende totalmente centrado en la autoatención para calcular las representaciones de su entrada y salida sin utilizar la secuencia RNN alineados o convolución. En las siguientes secciones, describiremos el Transformer, motive autoatención y discuta sus ventajas sobre modelos como [17, 18] y [9].

3 Arquitectura modelo

La mayoría de los modelos de transducción de secuencias neuronales competitivos tienen una estructura codificador-decodificado [2, 35]. En este caso, el codificador mapea una secuencia de entrada de representaciones de símbolos $x = (x_1, \dots, x_n)$ a una secuencia de representaciones continuas $z = (z_1, \dots, z_n)$. Dado z , el decodificador genera entonces una salida secuencia $(y_1, \dots, y_m)$ de símbolos, un elemento a la vez. En cada paso, el modelo es autorregresivo [10], consumiendo los símbolos generados anteriormente como entrada adicional al generar el siguiente.

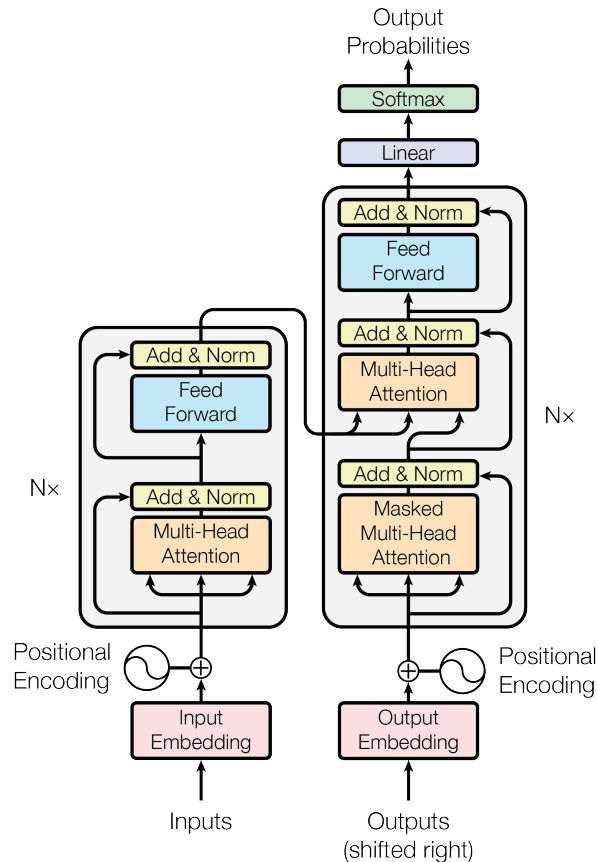


Figura 1: El Transformer: arquitectura del modelo.

El Transformer sigue esta arquitectura general utilizando una autoatención acumulada y puntual, totalmente capas conectadas tanto para el codificador como para el decodificador, mostradas en las mitades izquierda y derecha de la figura 1, respectivamente.

3.1 Pilas de codificadores y decodificadores

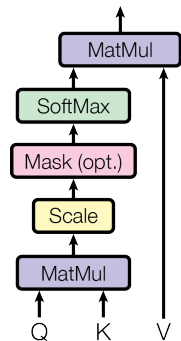
Codificador: El codificador está compuesto por una pila de $N = 6$ capas idénticas. Cada capa tiene dos subcapas. El primero es un mecanismo de autoatención con múltiples cabezales, y el segundo es un mecanismo simple, posicional sabia red de alimentación directa totalmente conectada. Empleamos una conexión residual [] alrededor de cada uno de las dos subcapas, seguidas de la normalización de capas []. Es decir, la salida de cada subcapa es $\text{Norma de capa} (x + \text{Subcapa} (x))$, donde $\text{Subcapa} (x)$ es la función implementada por la subcapa sí mismo. Para facilitar estas conexiones residuales, todas las subcapas del modelo, así como la incrustación capas, producen salidas de dimensión $d_{\text{modelo}} = 512$.

Decodificador: El decodificador también se compone de una pila de $N = 6$ capas idénticas. Además de las dos subcapas: en cada capa codificadora, el decodificador inserta una tercera subcapa, que realiza múltiples cabezales atención a la salida de la pila de codificadores. Al igual que en el codificador, empleamos conexiones residuales alrededor de cada una de las subcapas, seguida de la normalización de capas. También modificamos la autoatención subcapa en la pila de decodificadores para evitar que las posiciones presten atención a las posiciones posteriores. Esto el enmascaramiento, combinado con el hecho de que las incrustaciones de salida están compensadas en una posición, garantiza que predicciones de posición solo puede depender de las salidas conocidas en posiciones inferiores a i .

3.2 Atención

Una función de atención puede describirse como la asignación de una consulta y un conjunto de pares clave-valor a una salida, donde la consulta, las claves, los valores y la salida son todos vectores. La salida se calcula como una suma ponderada

Atención al producto Scaled Dot



Atención multicabezal

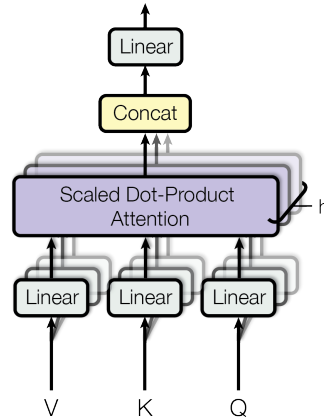


Figura 2: (izquierda) Scale Dot-Product Attention. (derecha) La atención con varios cabezas consiste en varias capas de atención que se ejecutan en paralelo.

de los valores, donde el peso asignado a cada valor se calcula mediante una función de compatibilidad del consulta con la clave correspondiente.

3.21 Atención al producto Scaled Dot

Llamamos nuestra atención en particular «Atención escalonada al producto» (Figura 2). La entrada consiste en consultas y claves de dimensión d_k , y valores de dimensión d_v . Calculamos los productos puntuales del consulta con todas las claves, divide cada una por d_k y aplique una función softmax para obtener los pesos de los valores.

En la práctica, calculamos la función de atención en un conjunto de consultas simultáneamente, agrupadas en una matriz Q . Las claves y los valores también se empaquetan en matrices K y V . Calculamos la matriz de salidas como:

$$\text{Atención} (Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

Las dos funciones de atención más utilizadas son la atención aditiva [1] y dot-product (multi-head) atención (replicativa). La atención al producto por puntos es idéntica a la de nuestro algoritmo, excepto por el factor de escala de $\frac{1}{\sqrt{d_k}}$. La atención aditiva calcula la función de compatibilidad mediante una red de retroalimentación con una sola capa oculta. Si bien los dos son similares en complejidad teórica, la atención al producto puntual es mucho más rápido y ahorra espacio en la práctica, ya que se puede implementar utilizando sistemas altamente optimizados de multiplicación de matrices.

Mientras que para valores pequeños los dos mecanismos funcionan de manera similar, la atención aditiva supera puntualmente la atención del producto sin escalar para valores más grandes. Sospechamos que para grandes valores de d_k los productos de puntos crecen a lo grande, lo que lleva la función softmax a las regiones en las que tiene gradientes extremadamente pequeños. Para contrarrestar este efecto, escalamos los productos de puntos de la siguiente manera $\frac{1}{\sqrt{d_k}}$.

3.22 Atención multicabezal

En lugar de realizar una única función de atención con d_{modelo} -claves dimensionales, valores y consultas, nos pareció beneficioso proyectar linealmente las consultas, las claves y los valores momentos en diferentes aprendidos proyecciones lineales a d_k , d_k y d_v dimensiones, respectivamente. En cada una de estas versiones proyectadas de consultas, claves y valores, luego realizamos la función de atención en paralelo, obteniendo d_v -dimensional

Para ilustrar por qué los productos con puntos aumentan de tamaño, supongamos que los componentes de y son independientes y aleatorios variables con media y varianza 0. Luego, su producto puntual, $q_k = \sum_{i=1}^n p_{ki} q_{ki}$, ha significado d_k varianza d_k .

valores de salida. Estos se concatenan y se proyectan una vez más, lo que da como resultado los valores finales, como representado en la figura 2.

La atención de múltiples cabezales permite al modelo atender de manera conjunta la información de diferentes representaciones subespacios en diferentes posiciones. Con un solo cabezal de atención, el promedio lo inhibe.

$$\text{Cabezal múltiple } (Q, K, V) = \text{Concat}(\text{cabeza}_1, \dots, \text{cabeza}_h) W^O$$

$$\text{donde } \text{cabeza}_i = \text{Atención}(QW_i^K, KW_i^K, V, W_i^V)$$

Donde las proyecciones son matrices de parámetros $W_i^Q \in \mathbb{R}^{d \times d}$, $W_i^K \in \mathbb{R}^{d \times d}$, $W_i^V \in \mathbb{R}^{d \times d}$ y $W^O \in \mathbb{R}^{hd \times d}$.

En este trabajo empleamos $h = 8$ capas de atención paralelas o cabezas. Para cada una de ellas utilizamos $d_k = d \Rightarrow d_{\text{modelo}}/h = 64$. Debido a la dimensión reducida de cada cabezal, el costo computacional total es similar a la de la atención con una sola cabeza con plena dimensionalidad.

3.23 Aplicaciones de la atención en nuestro modelo

El Transformer utiliza la atención con varios cabezales de tres maneras diferentes:

- En las capas de atención del codificador-decodificador, las consultas provienen de la capa de decodificación anterior, y las claves y valores de la memoria provienen de la salida del codificador. Esto permite que cada posición en el decodificador para atender todas las posiciones de la secuencia de entrada. Esto imita la mecanismos de atención típicos del codificador-decodificador en modelos de secuencia a secuencia, como [38, 2, 9].
- El codificador contiene capas de autoatención. En una capa de autoatención, todas las claves y valores y las consultas provienen del mismo lugar, en este caso, la salida de la capa anterior del codificador. Cada posición del codificador puede atender a todas las posiciones de la capa anterior del codificador.
- Del mismo modo, las capas de autoatención del decodificador permiten que cada posición del decodificador preste atención a todas las posiciones en el decodificador hasta esa posición, inclusive. Tenemos que prevenir hacia la izquierda flujo de información en el decodificador para preservar la propiedad autorregresiva. Implementamos esto dentro de la atención escalada del punto al producto enmascarando (configurando todos los valores de la entrada del softmax que corresponde a conexiones ilegales. Consulte la figura 2.

3.3 Redes de retroalimentación por posición

Además de las subcapas de atención, cada una de las capas de nuestro codificador y decodificador contiene un red de alimentación directa conectada, que se aplica a cada posición por separado e idénticamente. Esto consiste en dos transformaciones lineales con una activación de ReLU en el medio.

$$\text{FFN}(x) = \max(0, x - \text{Ancho} + 1b) \frac{W}{2} + b_2 \quad (2)$$

Si bien las transformaciones lineales son las mismas en diferentes posiciones, utilizan parámetros diferentes de capa en capa. Otra forma de describir esto es como dos convoluciones con un tamaño de núcleo de 1. La dimensionalidad de la entrada y la salida es $d_{\text{modelo}} = 512$, y la capa interna tiene dimensionalidad $d_{\text{ff}} = 2048$.

3.4 Embeddings y Softmax

De manera similar a otros modelos de transducción de secuencias, utilizamos incrustaciones aprendidas para convertir la entrada tokens y tokens de salida a vectores de dimensión d_{modelo} . También utilizamos la habitual transformación lineal aprendida Función mation y softmax para convertir la salida del decodificador en las probabilidades previstas para el siguiente token. En nuestro modelo, compartimos la misma matriz de peso entre las dos capas de incrustación y la anterior a softmax transformación lineal, similar a []. En las capas de incrustación, multiplicamos esos pesos por d_{modelo} .

Tabla 1: Longitudes máximas de ruta, complejidad por capa y número mínimo de operaciones secuenciales para diferentes tipos de capas. es la longitud de la secuencia, es la dimensión de representación, es el núcleo tamaño de las circunvoluciones y el tamaño del vecindario en una autoatención restringida.

Tipo de capa	Complejidad por capa	Secuencial Operaciones	Longitud máxima de ruta
Autoatención	$O(n^2)$	$O(1)$	$O(1)$
Recurrente	$O(nd)^2$	$O(n)$	$O(n)$
Convolucional	$O(knd)^2$	$O(1)$	$O(\text{registro}(n))$
Autoatención (restringida)	$O(rnd)$	$O(1)$	$O(n/r)$

3.5 Codificación posicional

Dado que nuestro modelo no contiene ninguna recurrencia ni convolución, para que el modelo utilice el orden de la secuencia, debemos inyectar alguna información sobre la posición relativa o absoluta del fichas en la secuencia. Para ello, añadimos «codificaciones posicionales» a las incrustaciones de entrada en partes inferiores de las pilas de codificadores y decodificadores. Las codificaciones posicionales tienen la misma dimensión como las incrustaciones, de modo que las dos puedan resumirse. Hay muchas opciones de codificaciones posicionales, aprendido y arreglado [9].

En este trabajo, utilizamos funciones de seno y coseno de diferentes frecuencias:

$$PE_{(\text{correo}, 2i)} = \text{pecado}(\text{pos}/10000^{2i/d})$$

$$PE_{(\text{correo}, 2i+1)} = \cos(\text{pos}/10000^{2i/d})$$

donde pos es la posición y es la dimensión. Es decir, cada dimensión de la codificación posicional corresponde a una senoide. Las longitudes de onda forman una progresión geométrica desde $10000^{2/d}$ a $10000^{2\pi}$. ¡Nosotros! elegimos esta función porque planteamos la hipótesis de que permitiría al modelo aprender fácilmente a asistir posiciones relativas, ya que para cualquier desfase fijo, $k PE_{\text{pos}+k}$ se puede representar como una función lineal de PE_{pos} .

También experimentamos con el uso de incrustaciones posicionales aprendidas [] en cambio, y descubrió que los dos las versiones produjeron resultados casi idénticos (véase la fila (E) de la Tabla 3). Elegimos la versión sinusoidal porque puede permitir que el modelo extrapole a longitudes de secuencia más largas que las encontradas durante el entrenamiento.

4 Por qué la autoatención

En esta sección comparamos varios aspectos de las capas de autoatención con las recurrentes y convolubles. capas cionales que se utilizan habitualmente para mapear una secuencia de representaciones de símbolos de longitud variable $(x_1, \dots, x_n)$ a otra secuencia de igual longitud $(z_1, \dots, z_n)$, con $x_i, z_i \in \mathbb{R}^d$, como un oculto capa en un codificador o decodificador de transducción de secuencias típico. Motivando nuestro uso de la autoatención, considera tres deseos.

Una es la complejidad computacional total por capa. Otra es la cantidad de computación que puede estar paralelizados, medidos por el número mínimo de operaciones secuenciales requeridas.

La tercera es la longitud de la ruta entre las dependencias de largo alcance de la red. Aprender a largo plazo las dependencias son un desafío clave en muchas tareas de transducción de secuencias. Un factor clave que afecta a la capacidad de aprender tales dependencias es la longitud de las rutas que las señales de avance y retroceso tienen que recorrer a través de la red. Cuanto más cortas sean estas rutas entre cualquier combinación de posiciones en la entrada y secuencias de salida, más fácil será aprender las dependencias de largo alcance []. Por lo tanto, también comparamos la longitud máxima de la ruta entre dos posiciones de entrada y salida en redes compuestas por diferentes tipos de capas.

Como se indica en la Tabla 1, una capa de autoatención conecta todas las posiciones con un número constante de operaciones ejecutadas, mientras que una capa recurrente requiere $O(n)$ operaciones secuenciales. En términos de complejidad computacional, las capas de autoatención son más rápidas que las capas recurrentes cuando la secuencia

longitud es menor que la dimensionalidad de la representación, que es el caso más frecuente de representaciones de oraciones utilizadas por modelos de última generación en traducciones automáticas, como word-piece [39] par de bytes [1] representaciones. Mejorar el rendimiento computacional para tareas que involucran secuencias muy largas, la autoatención podría restringirse a considerar solo un vecindario de tamaño en la secuencia de entrada centrada alrededor de la posición de salida respectiva. Esto aumentaría el máximo longitud de ruta hasta r . Planeamos investigar más a fondo este enfoque en trabajos futuros.

Una sola capa convolucional con ancho de núcleo $k < n$ no conecta todos los pares de entrada y salida posiciones. Hacerlo requiere una pila de $O(n/k)$ capas convolucionales en el caso de núcleos contiguos, o $O(\log(n))$ en el caso de convoluciones dilatadas [18], aumentando la longitud de las rutas más largas entre dos posiciones de la red. Las capas convolucionales son generalmente más caras que capas recurrentes, por un factor de k . Convoluciones separables [19], sin embargo, disminuyen la complejidad considerablemente para $d + n d) \cdot k^2$. Incluso con $k = n$, sin embargo, la complejidad de un separable la convolución es igual a la combinación de una capa de autoatención y una capa de retroalimentación puntual, el enfoque que adoptamos en nuestro modelo.

Como beneficio secundario, la autoatención podría producir modelos más interpretables. Inspeccionamos la distribución de la atención para de nuestros modelos y presente y analice ejemplos en el apéndice. No solo la atención individual las cabezas aprenden claramente a realizar diferentes tareas, muchas parecen exhibir un comportamiento relacionado con la sintáctica y estructura semántica de las oraciones.

5 Entrenamiento

En esta sección se describe el régimen de entrenamiento de nuestros modelos.

5.1 Datos de entrenamiento y procesamiento por lotes

Nos capacitamos en el conjunto de datos estándar inglés-alemán del WMT 2014, que consta de aproximadamente 4,5 millones pares de oraciones. Las oraciones se codificaron mediante la codificación por pares de bytes [3] que tiene una fuente compartida: vocabulario objetivo de alrededor de 37000 fichas. Para el inglés-francés, utilizamos el WMT, que es significativamente más grande Conjunto de datos inglés-francés de 2014 que consta de 36 millones de oraciones y fichas divididas en un fragmento de 32 000 palabras vocabulario [1]. 138 pares de oraciones se agruparon por lotes según la longitud de secuencia aproximada. Cada entrenamiento El lote contenía un conjunto de pares de frases que contenían aproximadamente 25 000 tokens fuente y 25 000 fichas de destino.

5.2 Hardware y programación

Entrenamos nuestros modelos en una máquina con 8 GPU NVIDIA P100. Para nuestros modelos básicos, utilizando los hiperparámetros descritos a lo largo del artículo, cada paso de entrenamiento duró aproximadamente 0,4 segundos. Nosotros entrenó los modelos base durante un total de 100 000 pasos o 12 horas. Para nuestros modelos grandes, (descritos en línea inferior de la tabla 3), el tiempo de la etapa fue de 1,0 segundos. Los modelos más grandes fueron entrenados para dar 300 000 pasos (3,5 días).

5.3 Optimizador

Usamos el optimizador Adam [20] con $\beta_1 = 0.9$, $\beta_2 = 0.98$ y $\epsilon = 10^{-9}$. Variamos el aprendizaje puntúa a lo largo de la formación, según la fórmula:

$$\text{latente} = d_{\text{modelo}}^{-0.5} \cdot \text{minuto}(\text{paso_num}^{-0.5}, \text{paso_num calentamiento pasos}^{-1.5}) \quad (3)$$

Esto corresponde a aumentar la tasa de aprendizaje de forma lineal para la primera $\text{calentamiento_pasos}$ pasos de entrenamiento, y reduciéndolo después proporcionalmente a la raíz cuadrada inversa del número de paso. Hemos usado $\text{calentamiento_pasos} = 4000$.

5.4 Regularización

Empleamos tres tipos de regularización durante la formación:

Tabla 2: El Transformer obtiene mejores puntuaciones en BLEU que los modelos anteriores de última generación en el Los exámenes newstest2014 de inglés a alemán e inglés a francés cuestan una fracción del costo de la capacitación.

modelo	AZUL		Costo de capacitación (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23,75			
DeepAtt + PoSunk [39]		39,2		$1.0 \cdot 10^{20}$
GNMT + RL [38]]	24,6	39,92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2 [9]	25,16	40,46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26,03	40,56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
DeepAtt + PoSunk Ensemble [39]		40,4		$8.0 \cdot 10^{20}$
Conjunto GNMT + RL [38]	26,30	41,16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
Conjunto ConvS2 [9]	26,36	41,29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformador (modelo base)	27,3	38,1	$3.3 \cdot 10^{18}$	$1.8 \cdot 10^{18}$
Transformador (grande)	28,4	41,8	$2.3 \cdot 10^{19}$	$1.9 \cdot 10^{19}$

Abandono residual Aplicamos la deserción [33] a la salida de cada subcapa, antes de añadirla a la entrada de subcapa y normalizada. Además, aplicamos la deserción a las sumas de las incrustaciones y las codificaciones posicionales tanto en las pilas de codificadores como de decodificadores. Para el modelo base, utilizamos una tasa de $P_{\text{soltar}} = 0.1$.

Suavizado de etiquetas Durante la capacitación, empleamos el suavizado de etiquetas de valor $\epsilon = 0.1$ [36] esto perjudica la perplejidad, ya que el modelo aprende a ser más inseguro, pero mejora la precisión y la puntuación BLEU.

6 Resultados

6.1 Traducción automática

En la tarea de traducción del inglés al alemán del WMT 2014, el gran modelo de transformador (Transformer (big) en la Tabla 2) supera a los mejores modelos reportados anteriormente (incluidos los conjuntos) en más de 2.0 BLEU, que establece una nueva puntuación BLEU de última generación de. La configuración de este modelo es enumerados en la línea inferior de la Tabla 3. La formación dura después 8 GPU P100. Incluso nuestro modelo base supera todos los modelos y conjuntos publicados anteriormente, a una fracción del costo de capacitación de cualquiera de los modelos competitivos.

En la tarea de traducción del inglés al francés del WMT 2014, nuestro gran modelo obtiene una puntuación BLEU de 40.8 , superando a todos los modelos individuales publicados anteriormente, a menos de $1/4$ el costo de formación del modelo anterior de última generación. Se utilizó el modelo Transformer (big), entrenado del inglés al francés tasa de deserción $P_{\text{soltar}} = 0.1$, en lugar de 0.3 .

Para los modelos base, utilizamos un solo modelo obtenido al promediar los últimos 5 puntos de control, que se escribieron a intervalos de 10 minutos. Para los modelos grandes, promediamos los últimos 20 puntos de control. Nosotros búsqueda de haz usada con un tamaño de haz de 4 y penalización por longitud $\alpha = 0.6$ [38] estos hiperparámetros fueron elegidos después de experimentar en el set de desarrollo. Establecemos la longitud máxima de salida durante inferencia a la longitud de entrada, pero terminan antes de lo posible [38].

En la tabla 2 se resumen nuestros resultados y se comparan la calidad de la traducción y los costes de formación con los de otros modelos arquitecturas de la literatura. Estimamos el número de operaciones de punto flotante utilizadas para entrenar a un modelo multiplicando el tiempo de entrenamiento, la cantidad de GPU utilizadas y una estimación de la duración sostenida capacidad de punto flotante de precisión única de cada GPU.⁵

6.2 Variaciones del modelo

Para evaluar la importancia de los diferentes componentes del Transformer, variamos nuestro modelo base de diferentes maneras, midiendo el cambio en el rendimiento de la traducción del inglés al alemán en el

⁵ Usamos valores de 2.8, 3.7, 6.0 y 9.5 TFLOPS para K80, K40, M40 y P100, respectivamente.

Tabla 3: Variaciones en la arquitectura del Transformer. Los valores no listados son idénticos a los de la base modelo. Todas las métricas figuran en el conjunto de desarrollo de traducciones del inglés al alemán, newstest2013. En la lista las perplejidades se producen por palabra, de acuerdo con nuestra codificación de pares de bytes, y no deben compararse con perplejidades por palabra.

	N	d _{modelo}	d _{ff}	h	d _k	d _v	P _{soltar}	ϵ _{ls}	tren pasos	PERSONAZUL (desarrollo)	parámetros (desarrollo) × 10 ⁶		
base	6	512	2048	8	64	64	0,1	0,1	100 MIL	4,92	25,8	65	
(A)										5,29	24,9		
										5,00	25,5		
										4,91	25,8		
										5,01	25,4		
(B)										5,16	25,1	58	
										5,01	25,4	60	
(C)	2										6,11	23,7	36
	4										5,19	25,3	50
	8										4,88	25,5	80
		256			32	32					5,75	24,5	28
		1024			128	128					4,66	26,0	168
			1024								5,12	25,4	53
			4096							4,75	26,2	90	
(D)										5,77	24,6		
										4,95	25,5		
										4,67	25,3		
										5,47	25,7		
(E)	incrustación posicional en lugar de sinusoides									4,92	25,7		
grande	6	1024	4096	16			0,3		300 MIL	4,33	26,4	213	

set de desarrollo, newtest 2013. Usamos la búsqueda por haz como se describe en la sección anterior, pero no promediando puntos de control. Presentamos estos resultados en la Tabla 3.

En las filas (A) de la Tabla 3, variamos el número de cabezas de atención y las dimensiones clave y valorativas de la atención, mantener constante la cantidad de cálculo, tal como se describe en la sección 3.2.2. Mientras sea de un solo cabezal La atención es 0.9 BLEU peor que la mejor configuración, la calidad también disminuye con demasiados cabezales.

En las filas de la Tabla 3 (B), observamos que reducir el tamaño de la clave de atención perjudica la calidad del modelo. Esto sugiere que determinar la compatibilidad no es fácil y que una compatibilidad más sofisticada función que el producto punto puede ser beneficioso. Observamos además en las filas (C) y (D) que, como era de esperar, Los modelos más grandes son mejores y el abandono es muy útil para evitar que se ajusten demasiado. En la fila (E) sustituimos nuestro codificación posicional sinusoidal con incrustaciones posicionales aprendidas [37], y observa casi idénticos resultados para el modelo base.

6.3 Análisis de distritos electorales en inglés

Para evaluar si el Transformer puede generalizarse a otras tareas, realizamos experimentos en inglés. análisis de circunscripciones. Esta tarea presenta desafíos específicos: la producción está sujeta a fuertes estructuras restringe y es significativamente más largo que la entrada. Además, RNN secuencia a secuencia los modelos no han podido obtener resultados de última generación en regímenes de datos pequeños [37].

Entrenamos un transformador de 4 capas con $d_{\text{modelo}} = 1024$ en la sección del Wall Street Journal (WSJ) del Penn Treebank [37], alrededor de 40 000 frases de entrenamiento. También lo entrenamos en un entorno semisupervizado, utilizando los corpus más grandes de alta confianza y BerkeleyParser de aproximadamente 17 millones de oraciones [37]. Usamos un vocabulario de 16 000 fichas solo para el escenario del WSJ y un vocabulario de 32 000 fichas para el entorno semisupervizado.

Realizamos solo un pequeño número de experimentos para seleccionar la deserción, tanto la atención como la residual. (sección 5.4), las tasas de aprendizaje y el tamaño del haz en el conjunto de desarrollo de la Sección 22, todos los demás parámetros se mantuvo sin cambios con respecto al modelo de traducción base del inglés al alemán. Durante la inferencia, nosotros

Tabla 4: El Transformer se generaliza bien al análisis de las circunscripciones en inglés (los resultados se encuentran en la Sección 23) del WSJ)

Analizador	Entrenamiento	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	Solo en el WSJ, discriminatorio	88,3
Petrov y otros (2006) [29]	Solo en el WSJ, discriminatorio	90,4
Zhu y otros (2013) [40]	Solo en el WSJ, discriminatorio	90,4
Dyer y otros (2016) [8]	Solo en el WSJ, discriminatorio	91,7
Transformador (4 capas)	Solo en el WSJ, discriminatorio	91,3
Zhu y otros (2013) [40]	semisupervisado	91,3
Huang y Harper (2009) [14]	semisupervisado	91,3
McClosky y otros (2006) [26]	semisupervisado	92,1
Vinyals & Kaiser et al. (2014) [37]	semisupervisado	92,1
Transformador (4 capas)	semisupervisado	92,7
Luong y otros (2015) [23]	multitarea	93,0
Dyer y otros (2016) [8]	generativo	93,3

aumentó la longitud máxima de salida a la longitud de entrada $\cdot 300$. Usamos un tamaño de viga d_v y $\alpha = 0.3$ solo para el WSJ y para el entorno semisupervisado.

Nuestros resultados en la Tabla 4 muestran que, a pesar de la falta de ajustes específicos para cada tarea, nuestro modelo funciona a la baja. sorprendentemente bien, ya que arroja mejores resultados que todos los modelos informados anteriormente, con la excepción del Gramática recurrente de redes neuronales [8].

A diferencia de los modelos RNN secuencia a secuencia [1], el Transformer supera al Berkeley-Analizador [29] incluso cuando se entrena solo con el conjunto de 40 000 oraciones de entrenamiento del WSJ.

7 Conclusión

En este trabajo, presentamos el Transformer, el primer modelo de transducción de secuencias basado completamente en atención, sustituyendo las capas recurrentes más utilizadas en las arquitecturas de codificador-decodificador por autoatención multifacética.

Para las tareas de traducción, el Transformer se puede entrenar significativamente más rápido que las basadas en arquitecturas en capas recurrentes o convolucionales. Tanto en el WMT 2014 de inglés a alemán como en el WMT 2014

En las tareas de traducción del inglés al francés, logramos un nuevo estado del arte. En la primera tarea, damos lo mejor de nosotros El modelo supera incluso a todos los conjuntos reportados anteriormente.

Estamos entusiasmados con el futuro de los modelos basados en la atención y planeamos aplicarlos a otras tareas. Nosotros planeamos extender el Transformer a problemas relacionados con modalidades de entrada y salida distintas del texto y investigar los mecanismos locales de atención restringida para gestionar de manera eficiente las grandes entradas y salidas como imágenes, audio y vídeo. Hacer que la generación sea menos secuencial es otro de nuestros objetivos de investigación.

El código que utilizamos para entrenar y evaluar nuestros modelos está disponible en <https://github.com/tensorflow/tensor2tensor>.

Agradecimientos Agradecemos a Nal Kalchbrenner y Stephan Gouws por su fructífera labor comentarios, correcciones e inspiración.

Referencias

- [1] Jimmy Lei Ba, Jamie Ryan Kiros y Geoffrey E Hinton. Normalización de capas. Preimpresión de arXiv Archivo IV: 1607.06450, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho y Yoshua Bengio. Traducción automática neuronal de forma conjunta aprender a alinear y traducir. CorR, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong y Quoc V. Le. Exploración masiva de lo neuronal arquitecturas de traducción automática. CorR, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong y Mirella Lapata. Redes de memoria a corto plazo para máquinas lectura. Preimpresión de arXiv ArXiv: 1601.06733, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Gulcehre Collobert, Fethi Bougares, Holger Schwenk y Yoshua Bengio. Representación de palabras en las redes de lenguaje recurrentes para fines estadísticos. *CorR* , abs/1406.1078, 2014.
- [6] François Chollet. Xception: Deep convolutional neural network with groupwise separable convolutions. *ArXiv Preprint arXiv:1610.02557* , 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho y Yoshua Bengio. Evaluación empírica de redes neuronales recurrentes cerradas en el modelado de secuencias. *CorR* , abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros y Noah A. Smith. Neural recurrent grammars of red. En *Proc. de la NAACL* , 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats y Yann N. Dauphin. Convolutional sequence-to-sequence learning. *Preimpresión de arXiv ArXiv: 1705.03122v2* , 2017. *Preimpresión de arXiv*
- [10] Alex Graves. Generación de secuencias con redes neuronales recurrentes. *ArXiv:1308.0850* , 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren y Jian Sun. Aprendizaje residual profundo para el reconocimiento de edad. En *Actas de la conferencia del IEEE sobre visión artificial y patrones Reconocimiento* , páginas 770—778, 2016.
- [12] Sébastien Hochreiter, Yoshua Bengio, Paolo Frasconi y Jürgen Schmidhuber. El flujo del gradiente hacia adentro en las redes recurrentes: la dificultad de aprender las dependencias a largo plazo, 2004.
- [13] Sébastien Hochreiter y Jürgen Schmidhuber. Memoria a corto plazo. *Computación neuronal* , 1997. 1735-1780, 1997.
- [14] Zhongqiang Huang y Mary Harper. Gramáticas PCFG de autoaprendizaje con anotaciones latentes en todos los idiomas. En *Actas de la conferencia de 2009 sobre métodos empíricos en la naturaleza Procesamiento del lenguaje* , páginas 832—841. ACL, agosto de 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer y Yonghui Wu. Explorando los límites del modelado del lenguaje. *Preimpresión de arXiv ArXiv: 1602.02410* , 2016. *Avances en el campo neuronal*
- [16] Łukasz Kaiser y Samy Bengio. ¿Puede la memoria activa reemplazar la atención? ¿En *Sistemas de procesamiento de información (NIPS)* , 2016.
- [17] Łukasz Kaiser e Ilya Sutskever. Las GPU neuronales aprenden algoritmos. En *Conferencia internacional sobre representaciones del aprendizaje (ICLR)* , 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves y Ko-Ray Kavukcuoglu. Traducción automática neuronal en tiempo lineal. *Preimpresión de arXiv ArXiv: 1610.10099v2* 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang y Alexander M. Rush. Redes de atención estructuradas. En *Conferencia Internacional sobre Representaciones del Aprendizaje 2017*.
- [20] Diederik Kingma y Jimmy Ba. Adam: Un método para la optimización estocástica. En *ICLR* , 2015.
- [21] Oleksii Kuchaiev y Boris Ginsburg. Trucos de factorización para redes LSTM. *Preimpresión de arXiv Archivo IV: 1703.10722* , 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicerón Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou y Yoshua Bengio. Una oración estructurada y atenta a sí misma que incorpora. *Preimpresión de arXiv Arxiv: 1703.03130* , 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals y Łukasz Kaiser. Tarea múltiple aprendizaje secuencia a secuencia. *Preimpresión de arXiv ArXiv: 1511.06114* , 2015.
- [24] Minh-Thang Luong, Hieu Pham y Christopher D. Manning. Enfoques efectivos de atención: traducción automática neuronal basada en atención. *Preimpresión de ArXiv ArXiv: 1508.04025* , 2015.

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz y Beatrice Santorini. Construyendo un gran anotado corpus de inglés: The Penn Treebank. *Lingüística computacional* , 19 (2) :313—330, 1993.
- [26] David McClosky, Eugene Charniak y Mark Johnson. Autoaprendizaje efectivo para analizar. En *Actas de la Conferencia sobre Tecnología del Lenguaje Humano de la NAACL, Conferencia principal* , páginas 152—159. ACL, junio de 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das y Jakob Uszkoreit. Una atención descomponible modelo. En *Métodos empíricos en el procesamiento del lenguaje natural* , 2016.
- [28] Romain Paulus, Caiming Xiong y Richard Socher. Un modelo de abstracción profundamente reforzado resumen. Preimpresión de arXiv ArXiv: 1705.04304 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux y Dan Klein. Aprendizaje preciso, compacto, y anotación de árbol interpretable. En *Actas de la 21ª Conferencia Internacional sobre Lingüística computacional y 44a reunión anual de la ACL* , páginas 433—440. ACL, julio 2006.
- [30] Ofir Press, Lior Wolf, Omer Levy y David Rosenberg. Incrustación de resultados para mejorar los modelos lingüísticos. ArXiv preprint arXiv:1608.06839 , 2016.
- [31] Rico Sennrich, Barry Haddow y Alexandra Birch. Traducción automática neuronal de palabras raras con unidades de subpalabras. Preimpresión de arXiv ArXiv: 1508.07909 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, y Jeff Dean. Redes neuronales escandalosamente grandes: una mezcla de expertos poco cerrada capa. Preimpresión de arXiv ArXiv: 1701.06538 2017.
- [33] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever y Ruslan Salakhutdinov. Abandono: una forma sencilla de evitar que las redes neuronales se sobreajusten. *Diario de máquinas Investigación sobre el aprendizaje* , 15 (1): 1929—1958, 2014.
- [34] Saizhuang Sukhbaatar, Arthur Szlam, Jason Weston y Bob Fergus. Memoria de extremo a extremo redes. En *C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama y R. Garnett, editores, Avances en los sistemas de procesamiento de información neuronal 28* , páginas 2440—2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals y Quoc VV Le. Aprendizaje secuencia a secuencia con neural redes. En *Avances en los sistemas de procesamiento de información neuronal* , páginas 3104—3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens y Zbigniew Wojna. Repensar la arquitectura inicial de la visión artificial. *CorR* , abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever y Hinton. La gramática como lengua extranjera. En *Avances en los sistemas de procesamiento de información neuronal* , 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey y col. La máquina neuronal de Google sistema de traducción: cerrar la brecha entre la traducción humana y la automática. Preimpresión de arXiv Archivo IV: 1609.08144 , 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li y Wei Xu. Modelos recurrentes profundos con conexiones rápidas para la traducción automática neuronal. *CorR* , abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang y Jingbo Zhu. Rápido y preciso análisis de constituyentes con reducción de tamaño. En *Actas de la 51ª reunión anual de la ACL (volumen 1: Artículos largos)* , páginas 434—443. ACL, agosto de 2013.

Visualizaciones de atención

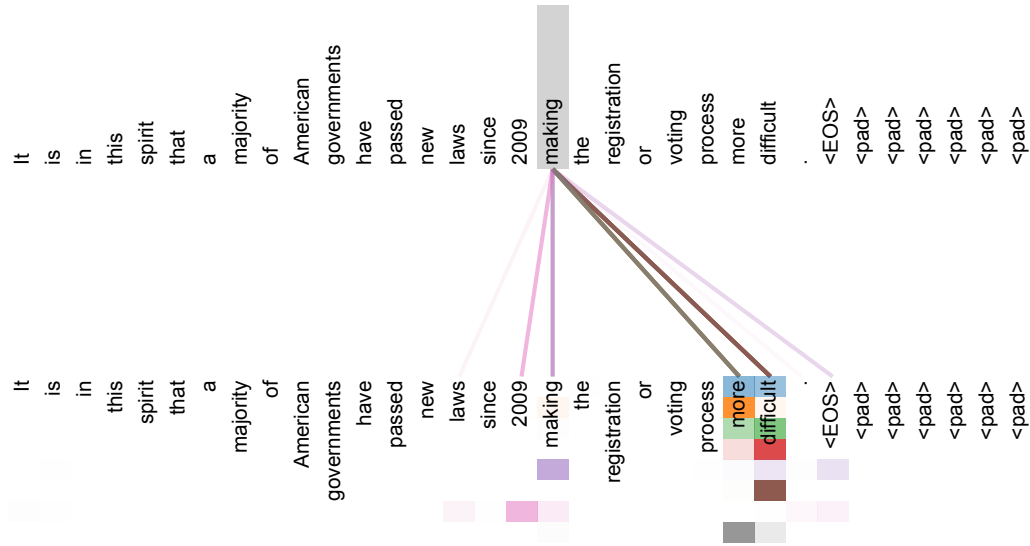


Figura 3: Un ejemplo del mecanismo de atención que sigue las dependencias a larga distancia en el autoatención del codificador en la capa 5 de 6. Muchos de los que llaman la atención atienden a una lejana dependencia de el verbo «hacer», completando la frase «hacer... más difícil». Las atenciones aquí se muestran solo para la palabra «hacer». Los diferentes colores representan diferentes cabezas. Se ve mejor en color.

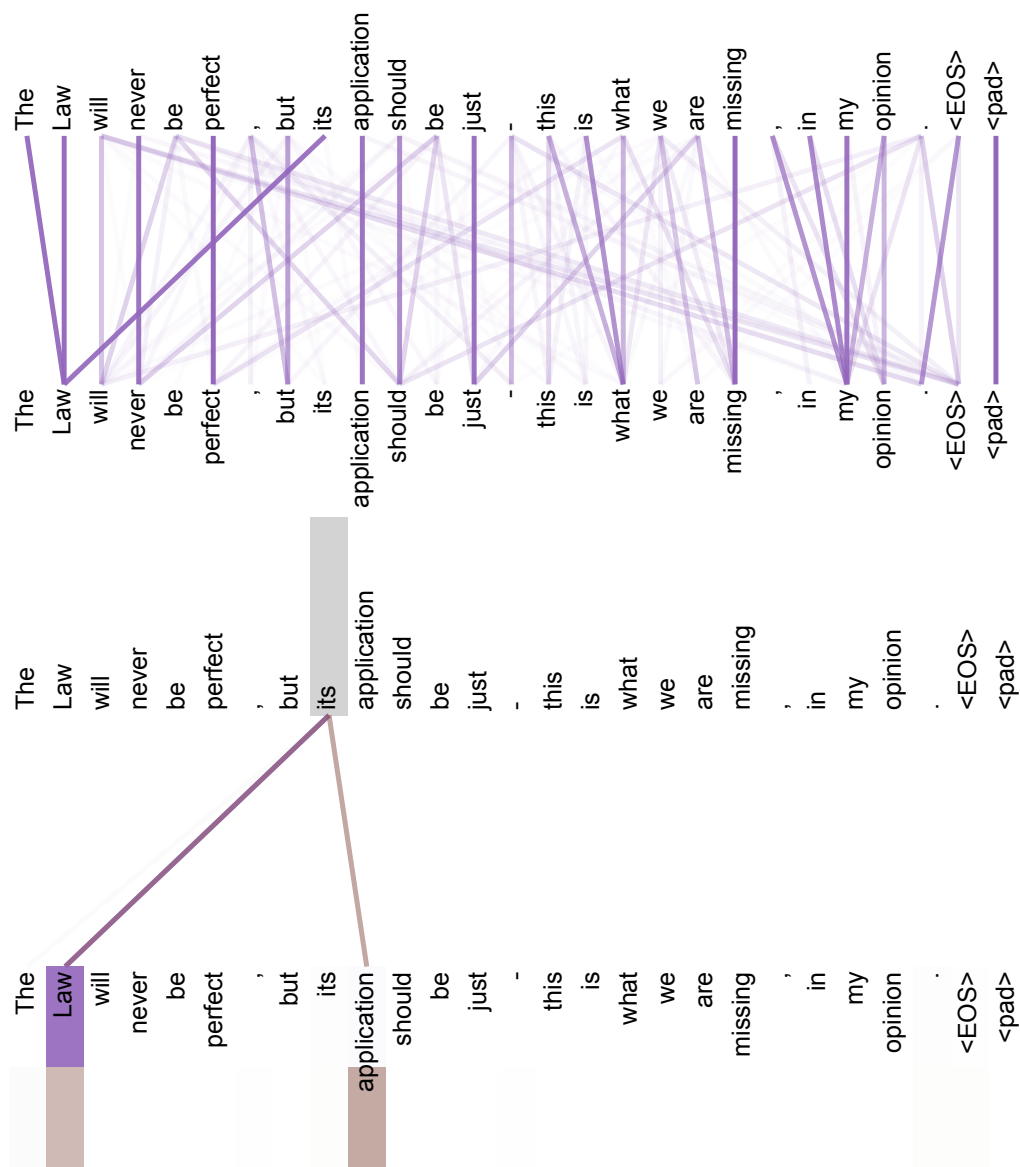


Figura 4: Dos cabezas de atención, también en la capa 5 de 6, aparentemente involucradas en la resolución de la anáfora. Parte superior: Atención completa para Head 5. Abajo: Atenciones aisladas de solo la palabra «es» para los que llaman la atención 5 y 6. Tenga en cuenta que se presta mucha atención a esta palabra.

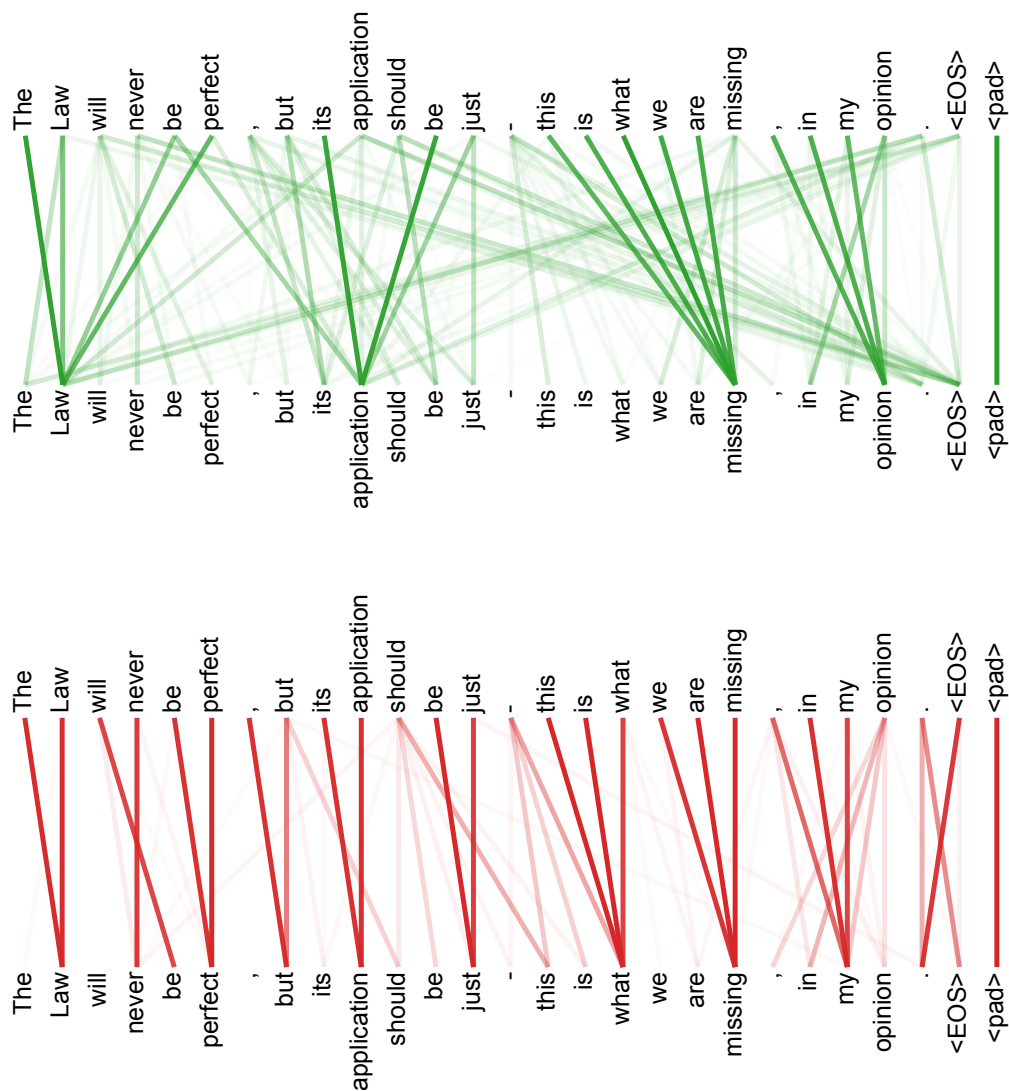


Figura 5: Muchos de los cabezas de atención muestran un comportamiento que parece estar relacionado con la estructura del *sentencia*. Damos dos ejemplos de este tipo anteriormente, de dos cabezales diferentes del codificador de autoatención en la capa 5 de 6. Los jefes claramente aprendieron a realizar diferentes tareas.